

15 Problem Statements — Built on the Engineering Project Blueprint Stack

Each problem statement is scoped so the *default* stack in the blueprint (Turborepo + pnpm, Next.js App Router, NestJS modular monolith, PostgreSQL + Prisma, Redis, BullMQ, Docker, GitHub Actions, OpenTelemetry, Terraform/OpenTofu) is the natural fit — not a stretch. Each one names the specific blueprint sections/patterns it forces you to actually implement (not just read about), since that's what makes the project defensible in an interview.

1. Multi-Tenant SaaS Invoicing & Billing Platform

Problem: Small businesses need to create customers, generate recurring invoices, accept payments, and see a real-time balance — with strict tenant data isolation. **Blueprint patterns exercised:** Shared-tables multi-tenancy (§8.8) with `tenant_id` scoping on every query, idempotency keys for payment webhooks (§7.5, §7.3), outbox pattern for "invoice paid → send receipt" (§8.5.1), NestJS modular structure (`billing`, `customers`, `payments` modules). **Stack:** Next.js (dashboard) + NestJS (`payments` module wraps a payment provider adapter) + PostgreSQL/Prisma + BullMQ (async receipt emails, PDF generation) + Redis (rate limit on payment endpoint). **What makes it resume-worthy:** You have to prove cross-tenant isolation with actual tests (§14.2), which is a specific, checkable claim most portfolios never demonstrate.

2. Marketplace Order & Fulfillment System

Problem: Buyers place orders against a multi-vendor catalog; inventory must decrement correctly under concurrent checkouts, and fulfillment status must sync from vendor systems. **Blueprint patterns exercised:** Explicit transactions for multi-write invariants (§8.5), bounded concurrency in workers (§7.3, §9.4), reconciliation job for ambiguous vendor-sync operations (§7.5), expand-and-contract migration when the order-state model changes mid-project (§8.3). **Stack:** NestJS `orders`/`inventory`/`fulfillment` modules, PostgreSQL row locking on inventory decrement, BullMQ for vendor-sync jobs with dead-letter handling, Terraform-provisioned Postgres + Redis. **What makes it resume-worthy:** Concurrent-checkout correctness is a classic system-design interview question — this project is the working proof, not a whiteboard answer.

3. Appointment Scheduling Platform with Timezone-Safe Booking

Problem: Service providers publish availability; clients book slots across timezones with no double-booking, plus reminder notifications and cancellation windows. **Blueprint patterns exercised:** Idempotent booking writes under retry (§7.3), scheduled work via managed scheduler → enqueued job, not app-level cron (§9.5), optimistic concurrency (ETags/version fields) to prevent double-booking races (§7.4.1). **Stack:** Next.js Server Components for availability display, NestJS `bookings` module, PostgreSQL unique constraint on `(provider_id, slot_start)`, BullMQ for reminder emails/SMS at scheduled offsets. **What makes it resume-worthy:** Timezone + concurrency correctness is a genuinely hard, well-understood class of bug — solving it visibly signals real engineering judgment.

4. Subscription Management & Dunning (Failed-Payment Recovery) System

Problem: Recurring subscriptions need automatic retry logic for failed payments, grace periods, and downgrade/cancellation state machines — without double-charging customers. **Blueprint patterns exercised:** Retry only transient, idempotent operations with backoff (§7.3, §9.4), state machine + outbox for cross-system workflows (§8.5), standardized error taxonomy for payment failures (§7.3). **Stack:** NestJS `subscriptions` module wrapping a payment-provider adapter, PostgreSQL state-machine table for subscription lifecycle, BullMQ scheduled dunning retries with exponential backoff, structured logs correlating retries via request/job IDs. **What makes it resume-worthy:** Dunning logic is exactly the kind of "boring but high-stakes" business logic that signals production maturity over demo polish.

5. Bulk CSV/Data Import & Export Pipeline

Problem: Users upload large CSV files (10K–1M rows) that need validation, transformation, and import into the primary database, with progress visibility and partial-failure handling. **Blueprint patterns exercised:** Object storage for uploads (§4.1), worker-based processing to protect request latency (§4.1, §9.3), bounded concurrency and job progress persistence (§9.4), safe payloads (store object keys, not raw data, in job metadata). **Stack:** Next.js upload UI with signed S3-compatible upload URLs, NestJS enqueues a job per file, BullMQ worker streams/validates/imports in batches, Redis for progress tracking exposed via polling endpoint. **What makes it resume-worthy:** Large-file async processing with real progress/partial-failure UX is a common "AI/data platform" interview probe — this project answers it directly.

6. Customer Support Ticketing System with SLA Tracking

Problem: Support tickets need routing, SLA-timer tracking, escalation on breach, and full audit trail of every status change. **Blueprint patterns exercised:** Event-based architecture for ticket state changes (§7.4.2), scheduled SLA-breach detection jobs (§9.5), structured audit logging (§16.1 audit control), standardized error/event taxonomy (`TicketEscalated`, `TicketResolved`). **Stack:** NestJS `tickets` module publishing domain events to an outbox, BullMQ consumer for SLA-timer checks, PostgreSQL for tickets + audit log tables, OpenTelemetry traces correlating a ticket's full lifecycle. **What makes it resume-worthy:** SLA/escalation logic is directly transferable to the "ops agent" AI projects in your other portfolio doc — this is the non-AI backbone that an agent could later plug into.

7. HR Onboarding & Compliance Workflow System

Problem: New hires move through a multi-step onboarding checklist (documents, approvals, provisioning) with role-based visibility and compliance sign-off tracking. **Blueprint patterns exercised:** Attribute-based authorization (tenant + role + resource state) (§16.2), workflow-as-state-machine with explicit failure behavior (§8.5), feature flags for gradually rolling out new onboarding steps with owner/expiry (§6.x, §22). **Stack:** Next.js authenticated app router routes, NestJS `onboarding`/`compliance` modules, PostgreSQL row-level tenant scoping, Prisma migrations tested against production-shaped data (§8.3). **What makes it resume-worthy:** Demonstrates authorization design beyond basic RBAC — attribute-based rules are what actual enterprise HR-tech products need.

8. E-Commerce Inventory Sync Across Multiple Vendor APIs

Problem: Inventory counts must stay accurate across your platform and 2–3 external vendor systems (simulated APIs), each with different reliability and rate limits. **Blueprint patterns exercised:** Adapter

pattern per external integration with timeout/retry/rate-limit handling (§7.5), reconciliation jobs for operations that "may complete after the client disconnects" (§7.5), circuit-breaker behavior on a failing provider. **Stack:** NestJS `integrations/vendor-a`, `integrations/vendor-b` adapters behind a common interface, BullMQ for rate-limited sync jobs, Redis for distributed locks during reconciliation, OpenTelemetry spans per external call. **What makes it resume-worthy:** Multi-provider reconciliation under partial failure is precisely the kind of "systems thinking" tech-giant interviewers probe for — directly complements your ScaleServe AI-infrastructure project.

9. High-Concurrency Event Ticketing & Waitlist System

Problem: A ticket drop needs to handle a concurrency spike fairly — no overselling, a fair queue/waitlist, and graceful degradation under load. **Blueprint patterns exercised:** Redis for coordination/locks under a measured, justified use case (§9.2), the full scalability escalation ladder (§9.1) — optimize queries first, decouple to queue, cache selectively, scale horizontally only if still needed, load testing critical paths before claiming it "scales" (§3.4). **Stack:** NestJS `checkout` module with Redis-backed distributed lock per ticket type, BullMQ for waitlist processing, k6/Locust load test producing real p95/p99 numbers under simulated drop traffic. **What makes it resume-worthy:** This is the single best "prove you understand scale" project in the list — real load-test numbers under a realistic spike scenario are rare and highly credible.

10. AI-Assisted Content Moderation Queue (Human-in-the-Loop)

Problem: User-generated content needs automated flagging (via an LLM or classifier call) with a human-review queue for ambiguous cases and full audit trail of every moderation decision. **Blueprint patterns exercised:** External AI provider wrapped as an adapter with timeout/retry (§7.5), worker-based async classification to protect request latency (§4.1), audit logging of privileged/moderation actions (§16.1), feature flag to adjust auto-flag confidence thresholds safely (§22). **Stack:** NestJS `moderation` module, BullMQ worker calling an LLM/classifier adapter, PostgreSQL for decisions + audit log, Next.js admin console for the human-review queue. **What makes it resume-worthy:** This is the cleanest bridge between your AI-portfolio projects and this engineering blueprint — an AI call embedded in a properly engineered production system, not a standalone notebook.

11. Financial Reporting & Read-Model Sync Service

Problem: Finance needs fast historical reporting queries that must not compete with or slow down live transactional traffic. **Blueprint patterns exercised:** Data warehouse / read-model introduction trigger (§8.2 — "operational queries harm production or analytics scale grows"), outbox-driven eventual sync from the system of record (§8.5.1), read replicas as a scale-the-data-layer step (§9.1 step 5), never using the reporting store as the primary source of truth. **Stack:** PostgreSQL primary + read replica or a dedicated reporting schema, BullMQ consumer processing outbox events into denormalized report tables, Next.js reporting dashboard querying the read model only. **What makes it resume-worthy:** Read/write separation for reporting is a specific, correctly-scoped scale project — shows judgment about *when* to add this complexity, not just that you can.

12. Multi-Region Deployment for a Data-Residency-Sensitive Product

Problem: A product serving EU and US customers needs to keep EU personal data in-region while sharing non-sensitive infrastructure, with an explicit failover story. **Blueprint patterns exercised:** The multi-

region decision framework (§10.6) — explicit data ownership, conflict resolution, and failover mode defined *before* implementation; Terraform module boundaries per environment (§11.3); this is one of the few blueprint sections that explicitly warns against doing this without a real trigger, so the project should document why the trigger (data residency) is genuine. **Stack:** Terraform/OpenTofu with per-region environment modules, PostgreSQL with region-scoped databases, a routing layer that pins EU users to EU infrastructure. **What makes it resume-worthy:** Extremely strong for German/EU employers specifically — data residency is a live, real concern there, and this project proves you understand the operational complexity, not just the compliance buzzword.

13. Feature-Flag-Driven Progressive Rollout Platform

Problem: Internal teams need to ship risky features behind flags, roll out progressively (percentage/cohort-based), and have flags cleaned up automatically rather than accumulating as permanent branches. **Blueprint patterns exercised:** Feature flags with explicit owner and removal date (§22 anti-pattern: "feature flags without cleanup"), canary/rolling deployment strategy comparison (§15.3), typed feature-flag package in the monorepo (`packages/feature-flags`, §5.1). **Stack:** NestJS flag-evaluation service, PostgreSQL flag-state storage, Next.js admin UI for flag management with mandatory expiry-date field, GitHub Actions job that fails CI if a flag exceeds its expiry without cleanup. **What makes it resume-worthy:** Directly demonstrates you understand delivery *process*, not just code — a differentiator for founding-engineer roles where you'll set these conventions yourself.

14. Audit-Logged Admin Console for a Regulated Fintech Product

Problem: Internal staff need an admin console to view and modify customer financial data, with every privileged action authorized, logged, and reversible. **Blueprint patterns exercised:** Full security baseline (§16.1) — MFA for privileged users, deny-by-default authorization, tested privilege-escalation cases (§16.2); audit trail for every privileged action (§16.1, §17.5); standardized, redacted structured logging that explicitly excludes sensitive fields (§12.5). **Stack:** Next.js admin console behind a separate auth boundary, NestJS `admin` module with attribute-based authorization, PostgreSQL append-only audit log table, OpenTelemetry traces tagged with actor identity for every privileged mutation. **What makes it resume-worthy:** Security-first admin tooling is exactly what fintech/regulated-industry employers (a strong visa-sponsoring segment) screen for directly.

15. Zero-Downtime Schema Migration Demonstration Project

Problem: A live product needs a breaking schema change (e.g., splitting a `name` column into `first_name`/`last_name`) deployed with zero downtime and full rollback safety. **Blueprint patterns exercised:** This is the one project built specifically to demonstrate §8.3 end-to-end — expand-and-contract migrations, testing migrations against production-shaped data, adding indexes concurrently, and proving a rollback plan actually works, not just documenting one. **Stack:** PostgreSQL + Prisma migrations run through the full sequence (add nullable columns → dual-write → backfill → cut over reads → drop old column in a later release), staged against a sanitized production-shaped snapshot, with a recorded before/after showing zero request errors during the migration window. **What makes it resume-worthy:** Almost no portfolio project explicitly proves zero-downtime migration safety with evidence — this is a small, sharp, highly specific project that answers a real, common production incident cause.

How to Use These With Your AI Portfolio

Projects **#10** (AI-Assisted Content Moderation) and **#6** (Support Ticketing with SLAs) are the strongest bridges to your existing AI-engineering portfolio — they let you show the *same* production-engineering discipline (idempotency, observability, audit trails) wrapping an actual AI/LLM call, which is precisely the "AI Engineer = software engineer + ML engineer" hybrid profile you're targeting.

If you only build 2–3 from this list, a strong minimal set is:

#9 (Ticketing/Waitlist under load) — proves scale judgment with real load-test numbers.

#10 (AI Moderation Queue) — ties this general-engineering blueprint directly to your AI portfolio.

#15 (Zero-downtime migration) — small, sharp, and rare — most candidates can't show this at all.